

ASSIGNMENT 01

End-to-End MLOps Pipeline for Heart Disease Prediction

Student Name	Parag Shetgaonkar
Student ID	2024AC05220
Email (WILP)	2024ac05220@wilp.bits-pilani.ac.in
Course	AIMLCZG523 – Machine Learning Operations
GitHub Repo	https://github.com/ParagSG/heart-disease-mlopsv1
API Endpoint	http://172.210.109.108/predict (AKS External IP)
Grafana Dashboard	http://20.75.254.127 (Grafana, admin / MLOps@2026)

1. Project Overview

This assignment implements a complete, production-grade MLOps pipeline for binary classification of heart disease using the UCI Heart Disease dataset. The pipeline covers the full machine-learning lifecycle: data ingestion and exploratory analysis, preprocessing, model training with experiment tracking, RESTful API serving, containerisation with Docker, automated CI/CD via GitHub Actions, cloud deployment on Azure Kubernetes Service (AKS), and live observability with Prometheus and Grafana.

1.1 Objectives

Apply MLOps best practices to a real-world medical classification problem.

Track experiments reproducibly with MLflow (parameters, metrics, artefacts, model versions).

Package the inference service as a Docker container and push to Azure Container Registry.

Automate linting, testing, training validation, and image publishing with GitHub Actions.

Deploy to Azure AKS with rolling updates and health probes.

Instrument the API with Prometheus metrics and visualise them in Grafana.

1.2 Technology Stack

Category	Tool / Version
Language	Python 3.11
ML Framework	scikit-learn 1.6.1
Experiment Tracking	MLflow 2.12.2
API Framework	FastAPI 0.111.0 + Uvicorn 0.29.0
Containerisation	Docker (multi-stage, python:3.11-slim)
Container Registry	Azure Container Registry (paragmlopsacr.azurecr.io)
CI/CD	GitHub Actions (3-job pipeline)
Orchestration	Azure Kubernetes Service – mlops-aks (East US)
Monitoring	Prometheus + Grafana (kube-prometheus-stack via Helm 3.21.2)
Version Control	Git / GitHub

2. Dataset & Exploratory Data Analysis (EDA)

2.1 Dataset Description

The Heart Disease UCI dataset (Cleveland subset) contains 303 patient records with 14 attributes (13 predictors + 1 binary target). The target variable indicates presence (1) or absence (0) of heart disease. The dataset was downloaded directly from the UCI ML Repository inside the Colab notebook to ensure reproducibility.

Dataset ready: (303, 14)

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	1
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0

Dataset loaded: 303 rows x 14 columns

--- Descriptive Statistics ---

	count	mean	std	min	25%	50%	75%	max
age	303.0	54.438944	9.038662	29.0	48.0	56.0	61.0	77.0
sex	303.0	0.679868	0.467299	0.0	0.0	1.0	1.0	1.0
cp	303.0	3.158416	0.960126	1.0	3.0	3.0	4.0	4.0
trestbps	303.0	131.689769	17.599748	94.0	120.0	130.0	140.0	200.0
chol	303.0	246.693069	51.776918	126.0	211.0	241.0	275.0	564.0
fbs	303.0	0.148515	0.356198	0.0	0.0	0.0	0.0	1.0
restecg	303.0	0.990099	0.994971	0.0	0.0	1.0	2.0	2.0
thalach	303.0	149.607261	22.875003	71.0	133.5	153.0	166.0	202.0
exang	303.0	0.326733	0.469794	0.0	0.0	0.0	1.0	1.0
oldpeak	303.0	1.039604	1.161075	0.0	0.0	0.8	1.6	6.2
slope	303.0	1.600660	0.616226	1.0	1.0	2.0	2.0	3.0
ca	299.0	0.672241	0.937438	0.0	0.0	0.0	1.0	3.0
thal	301.0	4.734219	1.939706	3.0	3.0	3.0	7.0	7.0
target	303.0	0.458746	0.499120	0.0	0.0	0.0	1.0	1.0

Descriptive statistics – mean age 54.4, thalach mean 149.6

Feature	Type	Description
age	Numeric	Age in years (mean: 54.4, range: 29-77)
sex	Categorical (0/1)	Sex (1=male, 0=female)
cp	Categorical (0-3)	Chest pain type
trestbps	Numeric	Resting blood pressure (mean: 131.7 mm Hg)
chol	Numeric	Serum cholesterol (mean: 246.7 mg/dl)
fbs	Categorical (0/1)	Fasting blood sugar > 120 mg/dl
restecg	Categorical (0-2)	Resting ECG results
thalach	Numeric	Maximum heart rate achieved (mean: 149.6)
exang	Categorical (0/1)	Exercise-induced angina
oldpeak	Numeric	ST depression induced by exercise (mean: 1.04)
slope	Categorical (0-2)	Slope of peak exercise ST segment
ca	Categorical (0-4)	Major vessels coloured by fluoroscopy (4 missing)
thal	Categorical (1-3)	Thalassemia type (2 missing)
target	Binary (0/1)	Heart disease: 0=absent, 1=present

2.2 Missing Value Analysis

Contrary to initial assumptions, the dataset does have a small number of missing values in two features:

ca: 4 missing values (1.3% of 303 records).

thal: 2 missing values (0.66% of 303 records).

All other features: complete with no missing values.

Strategy: mode imputation applied within the preprocessing pipeline for both features.



Missing Values (%) – ca: 1.3%, thal: 0.66%

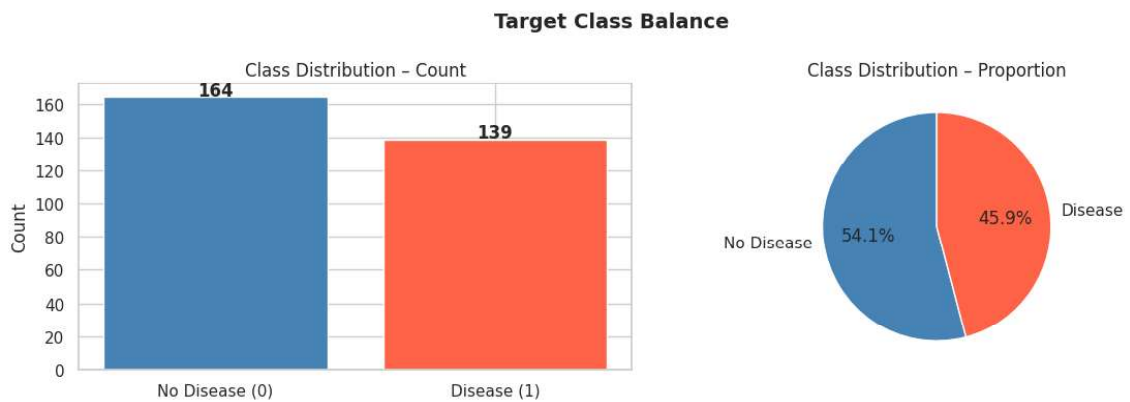
2.3 Class Distribution

The dataset is near-balanced, with a slight majority of No Disease cases:

No Disease (0): 164 records (54.1%)

Disease (1): 139 records (45.9%)

Near-balanced distribution — no resampling (SMOTE/oversampling) required.



Target Class Balance: 164 No Disease (54.1%) vs 139 Disease (45.9%)

2.4 Feature Analysis

Key findings from feature distribution analysis:

Thalach (max heart rate): Disease patients have significantly higher thalach values — strongest positive predictor (corr +0.42 with target).

Oldpeak (ST depression): Higher values strongly associated with no disease (+0.43 corr with target — note target encoding).

Age: Disease patients skew slightly younger (median ~58) vs no-disease patients (median ~53).

Cholesterol: High variance in both classes; weak discriminating power.

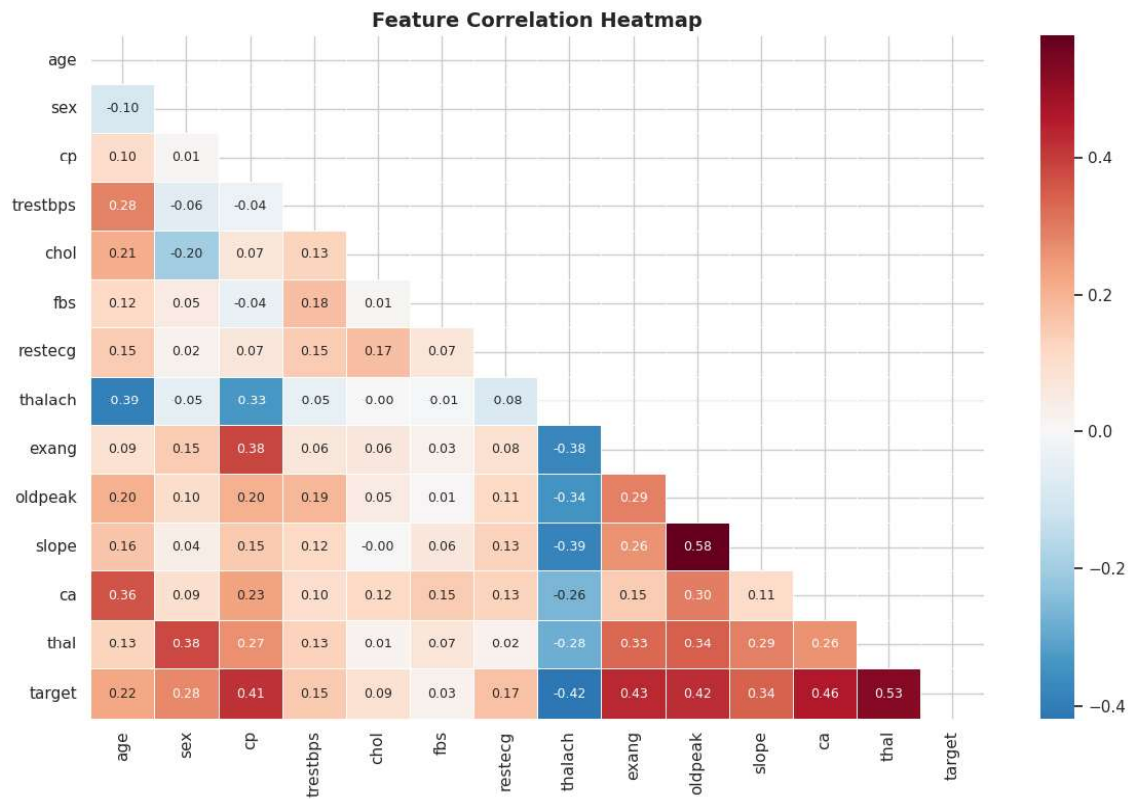
Exang (exercise-induced angina): Strong discriminating feature — most no-angina patients have disease.

cp (chest pain type=4): Most disease cases present with cp=4 (asymptomatic).

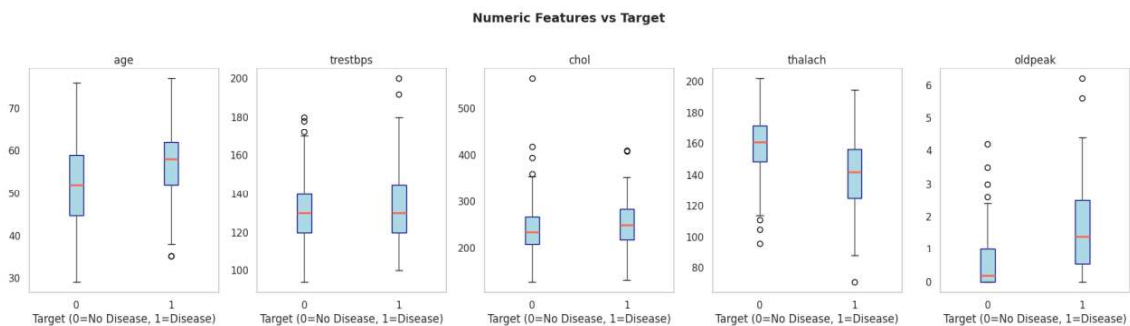
Thal=7 (reversible defect): Highest association with disease presence.



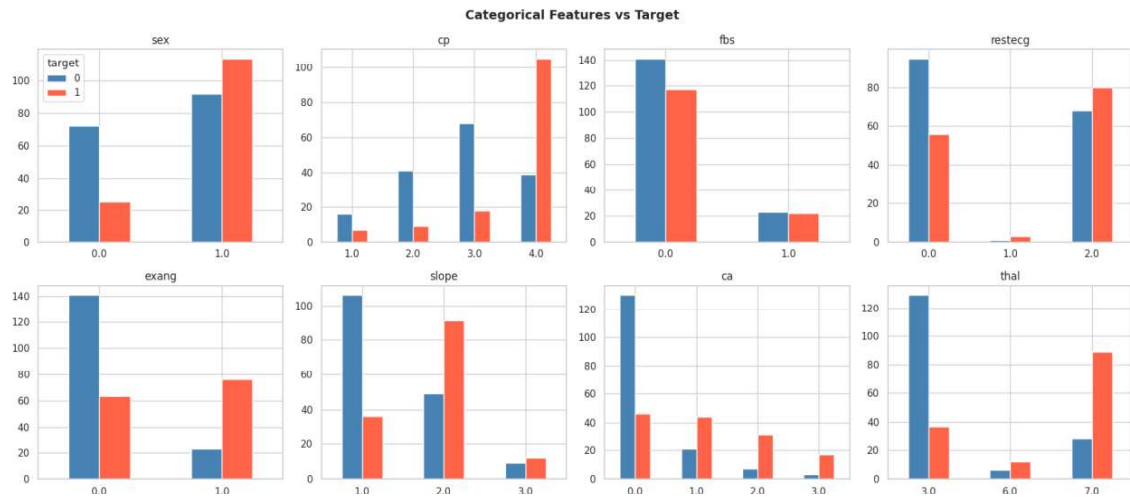
Numeric Feature Distributions by Target (age, trestbps, chol, thalach, oldpeak)



Feature Correlation Heatmap – thalach(-0.42), ca(+0.46), thal(+0.53) strongest with target



Numeric Features vs Target – thalach and oldpeak show clearest class separation



Categorical Features vs Target – cp, exang, thal, ca show strong class differences

3. Data Preprocessing

Preprocessing is encapsulated in a reusable sklearn Pipeline using ColumnTransformer (src/preprocess.py). This design eliminates training-serving skew: the identical transformation object is serialised alongside the model and loaded at inference time.

3.1 Pipeline Design

Numeric features (age, trestbps, chol, thalach, oldpeak): median imputation → StandardScaler.

Categorical features (sex, cp, fbs, restecg, exang, slope, ca, thal): mode imputation → OneHotEncoder (handle_unknown="ignore" for inference robustness).

Train/test split: 80/20 stratified split (random_state=42) preserving class balance.

The full sklearn Pipeline (preprocessor + classifier) serialised via joblib as models/best_model.joblib.

3.2 Code (src/preprocess.py)

```
numeric_features = ["age", "trestbps", "chol", "thalach", "oldpeak"]
categorical_features = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)])
```

4. Model Training & Experiment Tracking (MLflow)

Two classification models were trained with GridSearchCV hyperparameter tuning and tracked in MLflow under the experiment "heart-disease-classification". Training ran in Google Colab (src/train.py) with scikit-learn 1.6.1.

4.1 Logistic Regression

Hyperparameters searched: C in {0.01, 0.1, 1, 10}, class_weight in {None, "balanced"}.

5-fold stratified cross-validation, scoring=ROC-AUC.

Confusion matrix (test set): TN=28, FP=5, FN=2, TP=26.

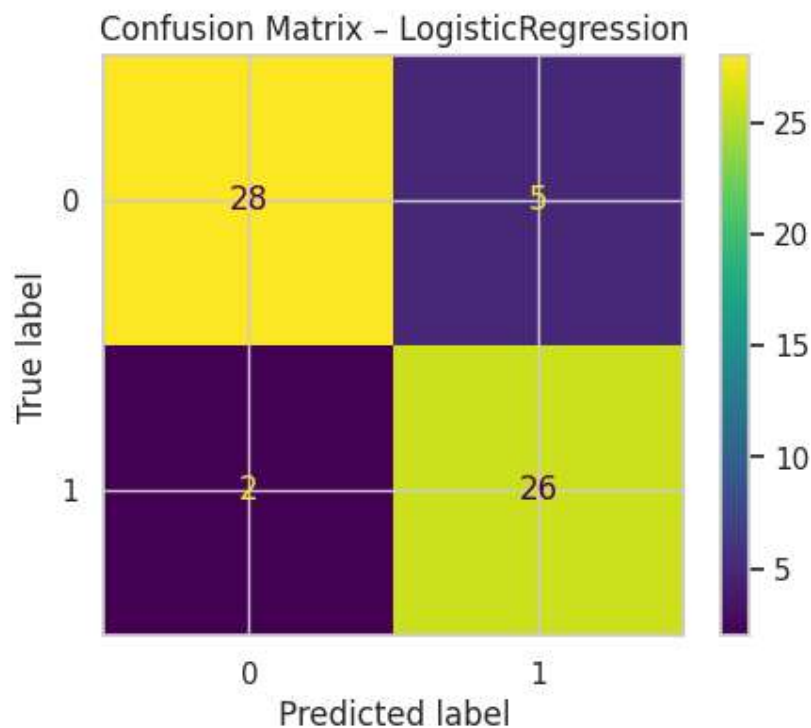
4.2 Random Forest

Hyperparameters searched: n_estimators in {100, 200}, max_depth in {None, 5, 10}, min_samples_split in {2, 5}.

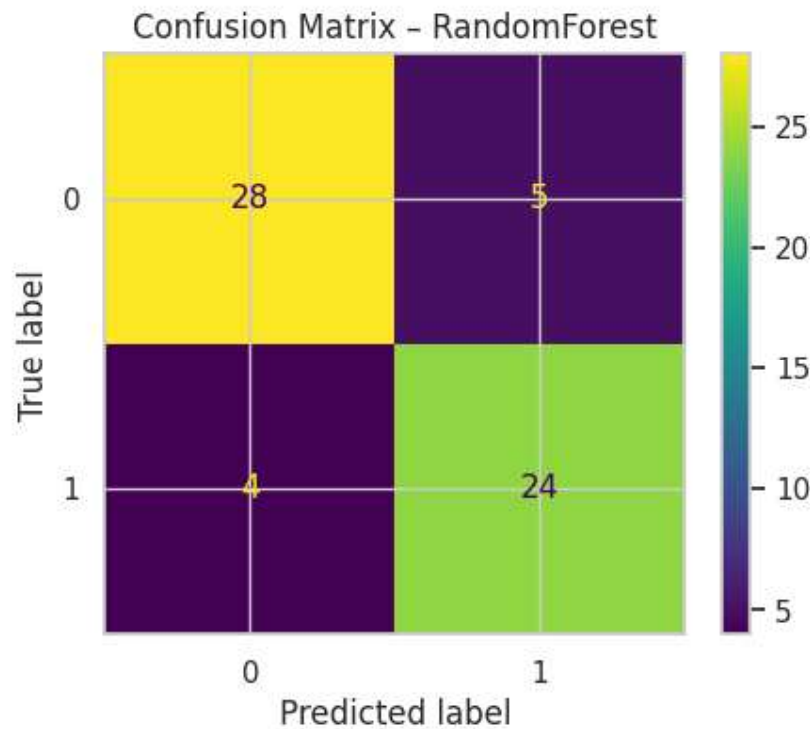
5-fold stratified cross-validation, scoring=ROC-AUC.

Confusion matrix (test set): TN=28, FP=5, FN=4, TP=24.

4.3 Confusion Matrices



Confusion Matrix – Logistic Regression (TN=28, FP=5, FN=2, TP=26)



Confusion Matrix – Random Forest (TN=28, FP=5, FN=4, TP=24)

4.4 Results Comparison

Metric	Logistic Regression	Random Forest
Accuracy	0.885 (54/61)	0.852 (52/61)
Precision	0.839	0.828
Recall	0.929	0.857
F1-Score	0.881	0.842
ROC-AUC	0.9654 (WINNER)	0.9470

Logistic Regression wins on all metrics. The trained sklearn Pipeline is saved as models/best_model.joblib.

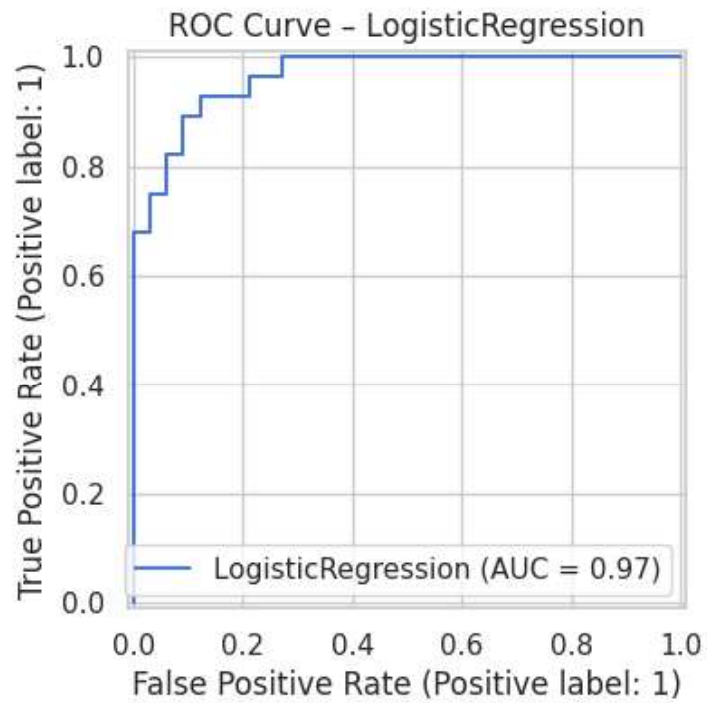
4.5 MLflow Tracking

Experiment name: "heart-disease-classification".

Logged per run: all hyperparameters, accuracy, precision, recall, F1, ROC-AUC.

Artefacts logged: confusion matrix PNG, ROC curve PNG, serialised sklearn Pipeline.

mlflow.sklearn.log_model() with skops_trusted_types for _RemainderColsList compatibility.



MLflow – Experiment runs comparing LR (AUC 0.9654) vs RF (AUC 0.9470)

	tags.mlflow.runName	params.model_type	metrics.roc_auc	metrics.accuracy	metrics.precision	metrics.recall	metrics.f1	
1	LogisticRegression	LogisticRegression	0.965368	0.885246	0.838710	0.928571	0.881356	
3	LogisticRegression	LogisticRegression	0.965368	0.885246	0.838710	0.928571	0.881356	
5	LogisticRegression	LogisticRegression	0.965368	0.885246	0.838710	0.928571	0.881356	
0	RandomForest	RandomForest	0.946970	0.852459	0.827586	0.857143	0.842105	
2	RandomForest	RandomForest	0.946970	0.852459	0.827586	0.857143	0.842105	
4	RandomForest	RandomForest	0.946970	0.852459	0.827586	0.857143	0.842105	

MLflow – Best run details: parameters, metrics, artefacts

5. API Development (FastAPI)

The inference service (src/app.py) is built with FastAPI and Uvicorn. The full sklearn Pipeline (preprocessor + model) is loaded at startup, eliminating training-serving skew.

5.1 Endpoints

Endpoint	Method	Description
GET /health	GET	Liveness/readiness probe. Returns {"status":"healthy","model_version":"1.0.0"}
POST /predict	POST	Accepts 13-feature JSON. Returns prediction (0/1), label, and confidence score.
GET /metrics	GET	Prometheus-format metrics scraped by kube-prometheus-stack.

5.2 Sample Request & Response

Request (from Azure Cloud Shell):

```
curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,
    "fbs":1,"restecg":0,"thalach":150,"exang":0,
    "oldpeak":2.3,"slope":0,"ca":0,"thal":1}'
```

Response:

```
{
  "prediction": 0,
  "label": "No Heart Disease",
  "confidence": 0.2958,
  "model_version": "1.0.0"
}
```

```
shetgaonkar [ ~ ]$ curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,
    "fbs":1,"restecg":0,"thalach":150,"exang":0,
    "oldpeak":2.3,"slope":0,"ca":0,"thal":1}'
{"prediction":0,"label":"No Heart Disease","confidence":0.3518,"model_version":"1.0.0"}shetgaonkar [ ~ ]$
```

FastAPI /predict endpoint test – successful JSON response

6. Containerisation with Docker

A multi-stage Dockerfile packages the inference service, serialised model, and all Python dependencies into a minimal production image (python:3.11-slim).

6.1 Dockerfile (Multi-Stage)

```
# Stage 1 - build dependencies
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Stage 2 - slim runtime image
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11/site-packages \
    /usr/local/lib/python3.11/site-packages
COPY src/ ./src/
COPY models/ ./models/
ENV MODEL_PATH=./models/best_model.joblib
EXPOSE 8000
CMD ["uvicorn", "src.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

6.2 Build, Test & Push

```
docker build -t heart-disease-api:latest .
```

```
docker run -p 8000:8000 heart-disease-api:latest
```

Smoke test: `curl http://localhost:8000/predict` — confirmed JSON response.

```
docker push paragmllopsacr.azurecr.io/heart-disease-api:latest
```

 — digest confirmed in ACR.

6.2 Local Docker Testing

The Docker image was built and run locally using Docker Desktop on Windows. The multi-stage build completed successfully (13/13 steps) and the container started with the model loaded, serving predictions on `http://localhost:8000`. Health checks returned HTTP 200 OK confirming the API was live.

```
docker build -t heart-disease-api:latest .
docker run -p 8000:8000 heart-disease-api:latest
```

```

Microsoft Windows [Version 10.0.22631.7219]
(c) Microsoft Corporation. All rights reserved.

D:\CLAUDE_working_directory\MLOPS\MLOPS\heart-disease-mlopsv1>az acr login --name paragalmlopsacr
'az' is not recognized as an internal or external command,
operable program or batch file.

D:\CLAUDE_working_directory\MLOPS\MLOPS\heart-disease-mlopsv1>docker build -t heart-disease-api:latest .
[+] Building 262.0s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.27kB
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> => transferring context: 2kB
=> [builder 1/6] FROM docker.io/library/python:3.11-slim@sha256:b27df5841f3355e9473f9a516d38a6783b6c8dfeaca2d14a24b4f43b368dddb
=> => resolve docker.io/library/python:3.11-slim@sha256:b27df5841f3355e9473f9a516d38a6783b6c8dfeaca2d14a24b4f43b368dddb
=> => sha256:ef18b70a77a2300080f50e00a11f6b3000e576f6970a304227770ecaf5ab 20MB / 20MB
=> => sha256:2bd111909f3359c1e4322cc60692041734884738cab078830e9af10d75a12f 14.36MB / 14.36MB
=> => sha256:d31bcade70704f7ea90d306d081a25dc35888777a7f904f232f85376bfff150 1.20MB / 1.20MB
=> => sha256:e95ade7ea7d00937920899023ec0b32796c9f6a1780401f6c0e5b06d13a 22.79MB / 22.79MB
=> => extracting sha256:e95ade7ea7d00937920899023ec0b32796c9f6a1780401f6c0e5b06d13a
=> => extracting sha256:d31bcade70704f7ea90d306d081a25dc35888777a7f904f232f85376bfff150
=> => extracting sha256:2bd111909f3359c1e4322cc60692041734884738cab078830e9af10d75a12f
=> => extracting sha256:ef18b70a77a2300080f50e00a11f6b3000e576f6970a304227770ecaf5ab
=> [internal] load build context
=> => transferring context: 19.20kB
=> [builder 2/6] WORKDIR /app
=> [builder 3/6] COPY requirements.txt .
=> [builder 4/6] RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
=> [stage-1 3/6] COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/site-packages
=> [stage-1 4/6] COPY --from=builder /usr/local/bin /usr/local/bin
=> [stage-1 5/6] COPY src/ ./src/
=> [stage-1 6/6] COPY models/ ./models/
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:8b5991507531f6ddf8bde21a33e072f958f9f0c91dce19f3b0993bc25eefc5
=> => exporting config sha256:64f6de2336085e4c09fca76d146e2319f40d1f4e0f80a0263360b0e857ab
=> => exporting attestation manifest sha256:b756351ec2f96a86e024d004f0b7bf55758802061ec5f6d97846c8c2726b1cha
=> => exporting manifest list sha256:79f138a11e9c0e9a07fde0ac33639038370fa08ec857c757a354f88ab027
=> => naming to docker.io/library/heart-disease-api:latest
=> => unpacking to docker.io/library/heart-disease-api:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/0n566r8gaxx6uhzd5e40f

```

Docker build 13/13 FINISHED

```

D:\CLAUDE_working_directory\MLOPS\MLOPS\heart-disease-mlopsv1>az acr repository list --name paragalmlopsacr --output table
'az' is not recognized as an internal or external command,
operable program or batch file.

D:\CLAUDE_working_directory\MLOPS\MLOPS\heart-disease-mlopsv1>docker run -p 8000:8000 heart-disease-api:latest
INFO:      Started server process [1]
INFO:      Waiting for application startup.
2026-07-06 14:16:57.177 | INFO | Model loaded from /app/models/best_model.joblib
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:      127.0.0.1:46690 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:39352 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:42228 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:40476 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:49532 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:38286 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:37174 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:33194 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:48414 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:57352 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:44088 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:52220 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:48930 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:52522 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:39234 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:59108 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:56020 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:33858 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:55304 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:44954 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:60588 - "GET / HTTP/1.0" 200 OK
INFO:      127.0.0.1:38464 - "GET /health HTTP/1.1" 200 OK
INFO:      172.17.0.1:34184 - "GET / HTTP/1.0" 200 OK
INFO:      172.17.0.1:34188 - "POST /sdk HTTP/1.1" 404 Not Found
INFO:      172.17.0.1:34200 - "GET /nmaplowercheck1783348827 HTTP/1.1" 404 Not Found
INFO:      172.17.0.1:34202 - "GET /evox/about HTTP/1.1" 404 Not Found
INFO:      172.17.0.1:34204 - "GET /HMAP1 HTTP/1.1" 404 Not Found
INFO:      172.17.0.1:34212 - "GET / HTTP/1.0" 200 OK
INFO:      172.17.0.1:34218 - "GET / HTTP/1.1" 200 OK
INFO:      127.0.0.1:34994 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:60560 - "GET /health HTTP/1.1" 200 OK
INFO:      127.0.0.1:38580 - "GET /health HTTP/1.1" 200 OK

```

Docker container running — model loaded, Uvicorn on 0.0.0.0:8000, GET /health 200 OK

7. CI/CD Pipeline (GitHub Actions)

The CI/CD pipeline (.github/workflows/ci.yml) runs automatically on every push to the main branch. Three sequential jobs must all pass before the Docker image is published to Azure Container Registry.

Job 1 – Lint + Unit Tests

Runs flake8 for PEP-8 compliance.

Executes pytest unit tests (tests/test_app.py).

Pipeline halts if linting or tests fail.

Job 2 – Model Training Validation

Downloads UCI dataset, runs src/train.py.

Verifies models/best_model.joblib is non-zero.

Uploads artefact for Job 3.

Job 3 – Docker Build Validation

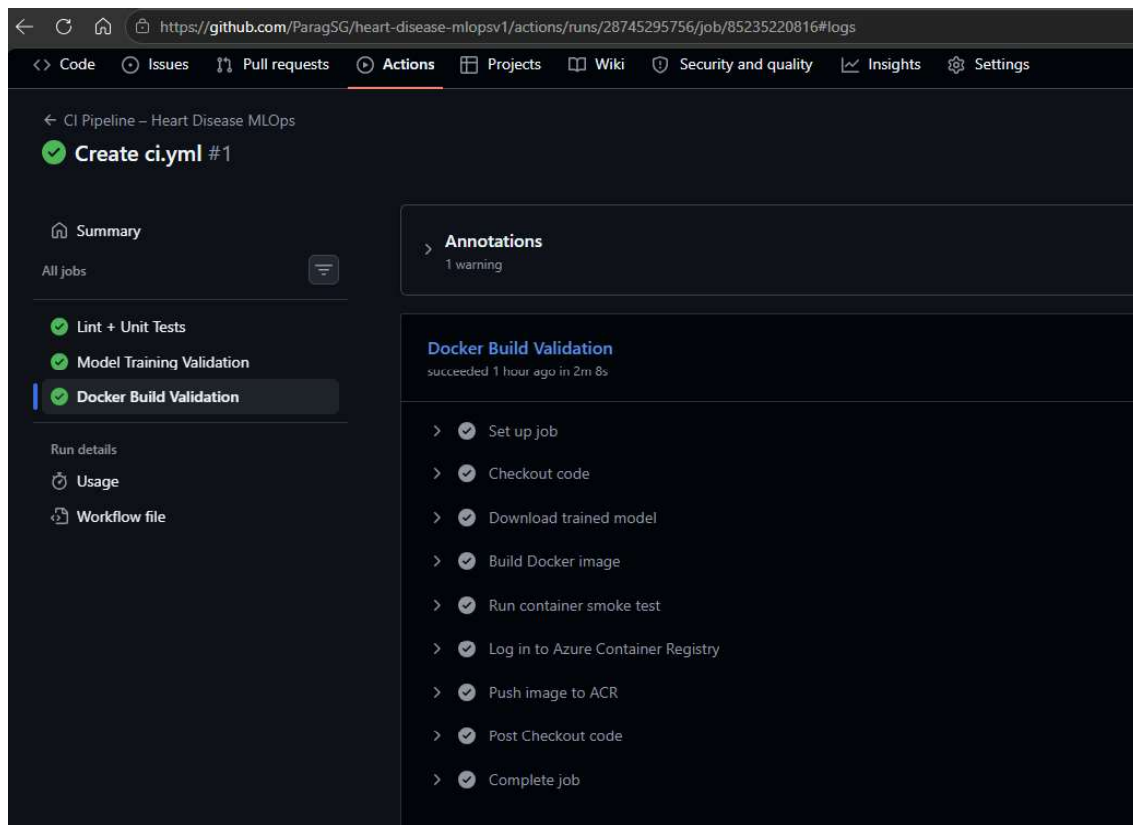
Builds Docker image, runs smoke test on /predict.

Logs in to ACR (GitHub Secrets).

Pushes image to paragmlopsacr.azurecr.io (main branch only).

The screenshot shows the GitHub Actions interface for the workflow 'CI Pipeline - Heart Disease MLOps'. The left sidebar contains navigation links: Code, Issues, Pull requests, Actions (selected), Projects, Wiki, Security and quality, Insights, and Settings. Under 'Actions', there are links for 'All workflows', 'CI Pipeline - Heart Disease MLOps', 'Management', 'Caches', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main area displays 'All workflows' with a search bar and a table of workflow runs. The table has columns for Workflow, Event, Status, Branch, and Actor. It shows 10 workflow runs, all of which are 'Add files via upload' events triggered by pushes to the 'main' branch. The runs are listed in descending order of time, with the most recent run at the top. Each run entry includes a status icon (green for success, red for failure), a description of the event, the commit hash, the actor's name, and the time taken for the run. The most recent run, triggered by commit '5a25e77' pushed by 'ParagSG', is marked as successful and took 4m 48s to complete.

Workflow	Event	Status	Branch	Actor
Add files via upload	CI Pipeline - Heart Disease MLOps #10: Commit 5a25e77 pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #9: Commit 6078d3a pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #8: Commit 5ea6334 pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #7: Commit aac4825 pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #6: Commit 4c790a6 pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #5: Commit d133d6d pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #4: Commit 6b46bb6 pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #3: Commit 9a43e77 pushed by ParagSG	Success	main	ParagSG
Add files via upload	CI Pipeline - Heart Disease MLOps #2: Commit 3a76644 pushed by ParagSG	Success	main	ParagSG



8. Kubernetes Deployment (Azure AKS)

The containerised API is deployed to Azure Kubernetes Service (AKS) in East US. Two replicas provide high availability with health probes and a LoadBalancer Service for public access.

8.1 Infrastructure

Resource	Value
Resource Group	mlops-rg
AKS Cluster	mlops-aks (East US, Standard_D2ds_v7)
Container Registry	paragmlopsacr.azurecr.io
Replicas	2
Service Type	LoadBalancer
External IP	172.210.109.108 (port 80 -> 8000)

8.2 Key Configuration

Liveness probe: GET /health, initialDelaySeconds=30, periodSeconds=10.

Readiness probe: GET /health, initialDelaySeconds=15, periodSeconds=5.

Rolling update: maxUnavailable=0, maxSurge=1 (zero-downtime deploys).

8.3 Verification

kubectl get pods -n default → 2/2 Running, 0 Restarts.

kubectl get svc → External-IP: 172.210.109.108.

curl http://172.210.109.108/predict → {"prediction":0,"label":"No Heart Disease","confidence":0.2958}

```
shetgaonkar [ ~ ]$ az aks get-credentials --resource-group mlops-rg --name mlops-aks
kubectl get pods
Merged "mlops-aks" as current context in /home/shetgaonkar/.kube/config
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-8558d55667-mcmgr  1/1     Running   0           4d22h
heart-disease-api-8558d55667-wlkq6  1/1     Running   0           4d22h
shetgaonkar [ ~ ]$
```

AKS – kubectl get pods showing 2 Running heart-disease-api pods

```
shetgaonkar [ ~ ]$ curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d '{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,"fbs":1,"restecg":0,"thalach":150,"exang":0,"oldpeak":2.3,"slope":0,"ca":0,"thal":1}'
{"prediction":0,"label":"No Heart Disease","confidence":0.3518,"model_version":"1.0.0"}shetgaonkar [ ~ ]$
```

AKS – curl /predict from Cloud Shell returning successful JSON

Example:

Case 1 — Young female, low risk (expect: No Disease):

```
curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d
'{"age":35,"sex":0,"cp":1,"trestbps":120,"chol":200,"fbs":0,"restecg":0,"thalach":170,"exang":0,"oldpeak":0.5,"slope":1,"ca":0,"thal":2}'
```

Case 2 — Older male, high risk (expect: Heart Disease):

```
curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
```

-d

```
'{"age":67,"sex":1,"cp":4,"trestbps":160,"chol":286,"fbs":0,"restecg":2,"thalach":108,"exang":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":3}'
```

Case 3 — Middle-aged male, borderline (expect: varies):

```
curl -X POST http://172.210.109.108/predict \
```

```
-H "Content-Type: application/json" \
```

-d

```
'{"age":52,"sex":1,"cp":2,"trestbps":135,"chol":250,"fbs":0,"restecg":1,"thalach":140,"exang":0,"oldpeak":1.2,"slope":1,"ca":1,"thal":2}'
```

```
Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.
shetgaonkar [ ~ ]$ az aks start --resource-group mlops-rg --name mlops-aks
shetgaonkar [ ~ ]$ az aks get-credentials --resource-group mlops-rg --name mlops-aks
kubectl get pods
Merged "mlops-aks" as current context in /home/shetgaonkar/.kube/config
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-8558d55667-7rcgl  1/1     Running   0           4d6h
heart-disease-api-8558d55667-n54v5  1/1     Running   0           4d6h
shetgaonkar [ ~ ]$ curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d '{"age":35,"sex":0,"cp":1,"trestbps":120,"chol":200,"fbs":0,"restecg":0,"thalach":170,"exang":0,"oldpeak":0.5,"slope":1,"ca":0,"thal":2}'
{"prediction":0,"label":0,"model_version":"1.0.0"}shetgaonkar [ ~ ]$ curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d '{"age":67,"sex":1,"cp":4,"trestbps":160,"chol":286,"fbs":0,"restecg":2,"thalach":108,"exang":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":3}'
{"prediction":1,"label":1,"model_version":"1.0.0"}shetgaonkar [ ~ ]$ curl -X POST http://172.210.109.108/predict \
-H "Content-Type: application/json" \
-d '{"age":52,"sex":1,"cp":2,"trestbps":135,"chol":250,"fbs":0,"restecg":1,"thalach":140,"exang":0,"oldpeak":1.2,"slope":1,"ca":1,"thal":2}'
{"prediction":1,"label":"Heart Disease Detected","confidence":0.5727,"model_version":"1.0.0"}shetgaonkar [ ~ ]$
```

9. Monitoring – Prometheus & Grafana

Observability is implemented using the kube-prometheus-stack Helm chart deployed to the monitoring namespace on AKS. The FastAPI /metrics endpoint is instrumented with prometheus-fastapi-instrumentator; Prometheus scrapes it automatically.

9.1 Deployment

Helm 3.21.2 installed to ~/.local/bin in Azure Cloud Shell.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

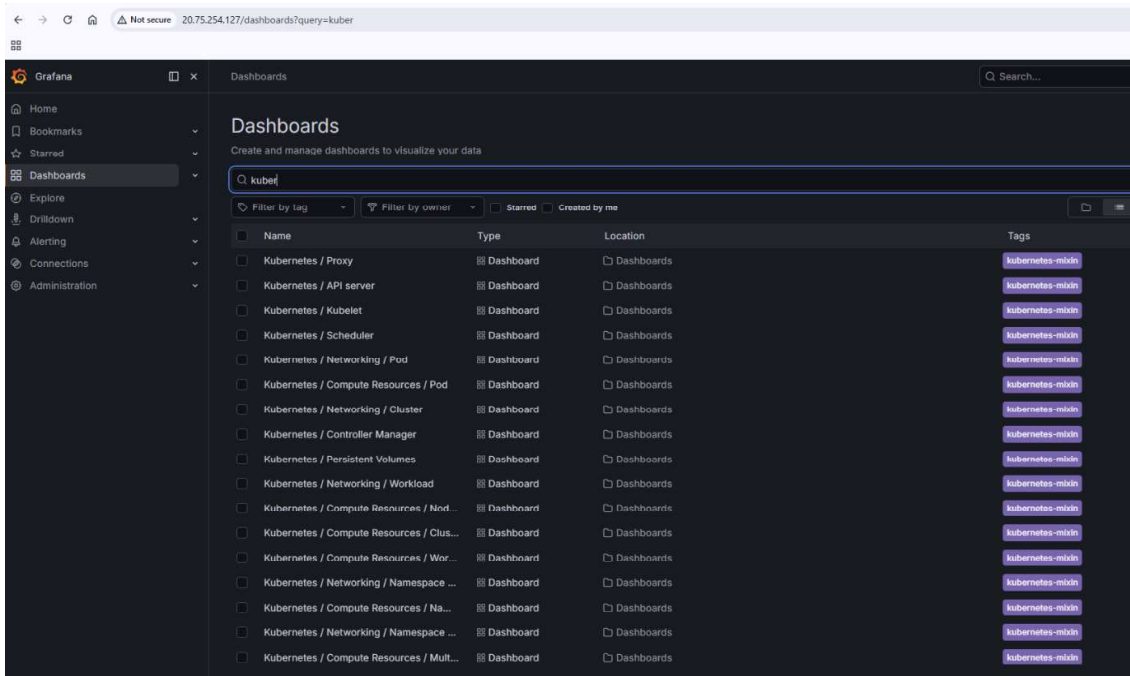
```
helm install monitoring prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespace --set grafana.adminPassword=MLOps@2026
```

All 6 monitoring pods confirmed Running.

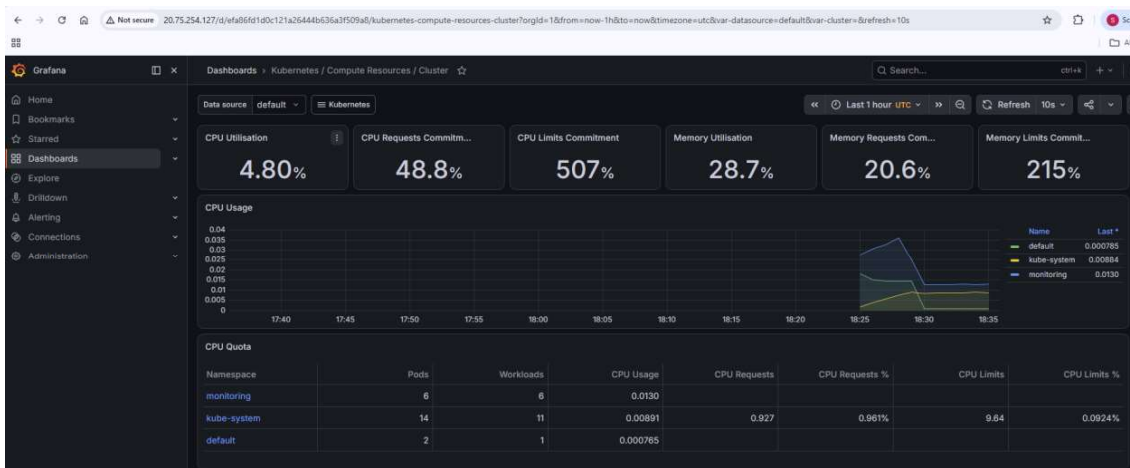
Grafana LoadBalancer External IP: 20.75.254.127 — accessed at <http://20.75.254.127> (admin / MLOps@2026).

9.2 Observed Metrics

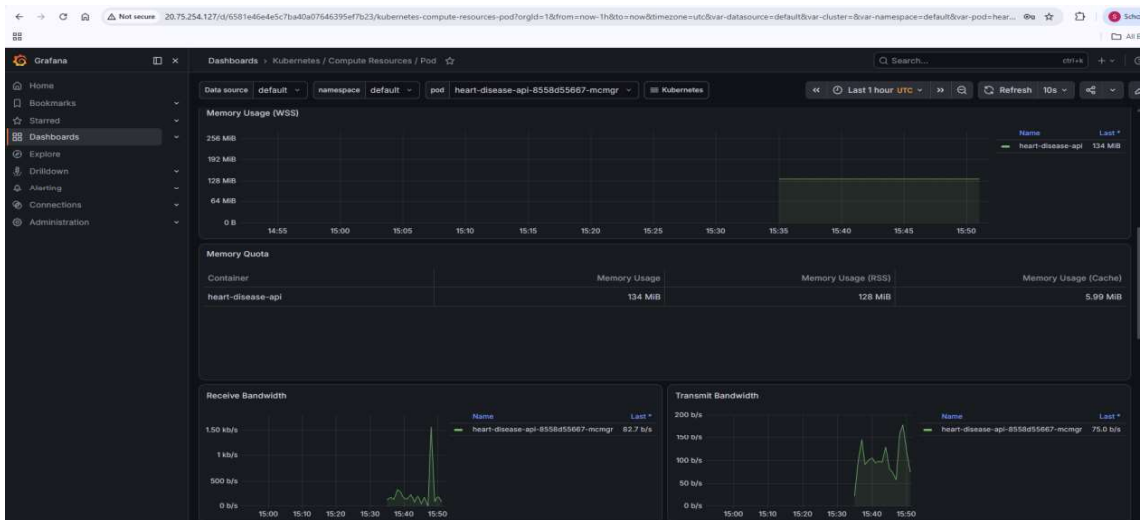
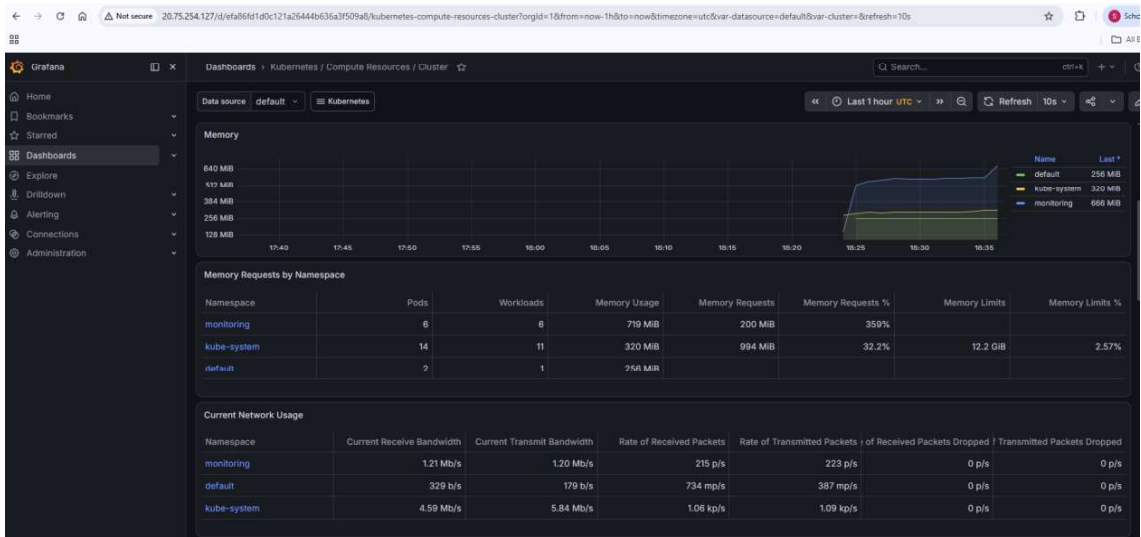
Metric	Observed Value
http_requests_total (GET /)	1 (status 2xx)
process_resident_memory_bytes	221 MB
process_cpu_seconds_total	3.81 CPU-seconds
python_info	CPython 3.11.15
Pod CPU Usage	0.000533 cores (idle)
Pod Memory (WSS)	134 MiB (RSS 128 MiB, Cache 5.99 MiB)
Receive Bandwidth	82.7 b/s
Transmit Bandwidth	75.0 b/s



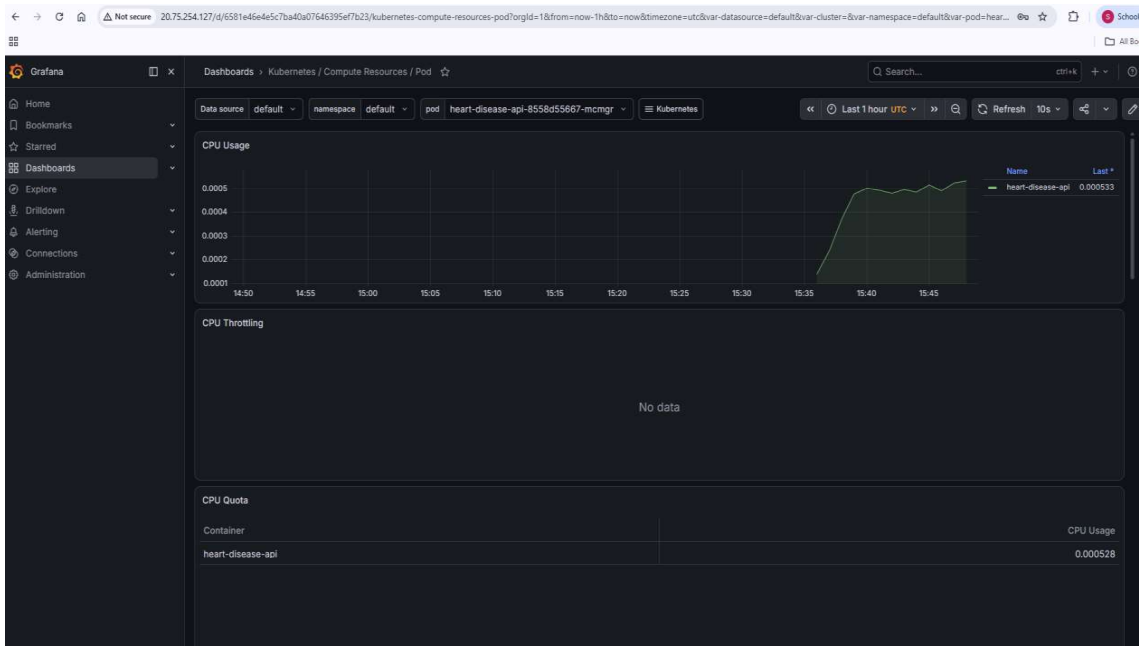
Grafana – 18 pre-built Kubernetes dashboards (kube-prometheus-stack)



Grafana – CPU Usage for heart-disease-api pod (0.000533 cores)



Grafana – Memory Usage + Receive/Transmit Bandwidth



```
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total(handler="/predict",method="POST",status="2xx") 1.0
# HELP http_requests_created Total number of requests by method, status and handler.
# TYPE http_requests_created gauge
http_requests_created(handler="/predict",method="POST",status="2xx") 1.78327057637286e+09
# HELP http_request_size_bytes Content length of incoming requests by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_request_size_bytes summary
http_request_size_bytes_count(handler="/predict") 1.0
http_request_size_bytes_sum(handler="/predict") 135.0
# HELP http_request_size_bytes_created Content length of incoming requests by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_request_size_bytes_created gauge
http_request_size_bytes_created(handler="/predict") 1.7832705763728807e+09
# HELP http_response_size_bytes Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes summary
http_response_size_bytes_count(handler="/predict") 1.0
http_response_size_bytes_sum(handler="/predict") 87.0
# HELP http_response_size_bytes_created Content length of outgoing responses by handler. Only value of header is respected. Otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes_created gauge
http_response_size_bytes_created(handler="/predict") 1.7832705763729012e+09
# HELP http_request_duration_highr_seconds Latency with many buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_highr_seconds histogram
http_request_duration_highr_seconds_bucket(le="0.01") 0.0
http_request_duration_highr_seconds_bucket(le="0.025") 1.0
http_request_duration_highr_seconds_bucket(le="0.05") 1.0
http_request_duration_highr_seconds_bucket(le="0.075") 1.0
http_request_duration_highr_seconds_bucket(le="0.1") 1.0
http_request_duration_highr_seconds_bucket(le="0.25") 1.0
http_request_duration_highr_seconds_bucket(le="0.5") 1.0
http_request_duration_highr_seconds_bucket(le="0.75") 1.0
http_request_duration_highr_seconds_bucket(le="1.0") 1.0
http_request_duration_highr_seconds_bucket(le="1.5") 1.0
http_request_duration_highr_seconds_bucket(le="2.0") 1.0
http_request_duration_highr_seconds_bucket(le="2.5") 1.0
http_request_duration_highr_seconds_bucket(le="3.0") 1.0
http_request_duration_highr_seconds_bucket(le="3.5") 1.0
http_request_duration_highr_seconds_bucket(le="4.0") 1.0
http_request_duration_highr_seconds_bucket(le="4.5") 1.0
http_request_duration_highr_seconds_bucket(le="5.0") 1.0
http_request_duration_highr_seconds_bucket(le="7.5") 1.0
```

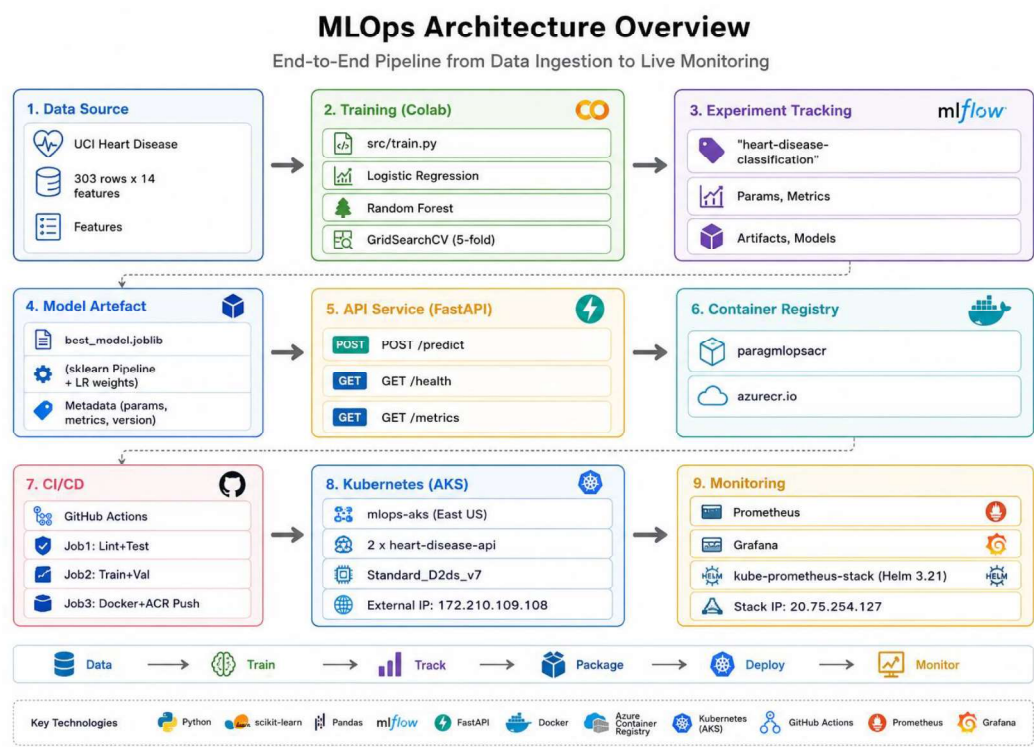
The image shows the 'Dashboards' page in Grafana. It features a search bar and a table of dashboards. The table has columns for Name, Type, Location, and Tags. The dashboards are filtered by tag 'kubernetes'.

Name	Type	Location	Tags
Kubernetes / Proxy	Dashboard	Dashboards	Kubernetes
Kubernetes / API server	Dashboard	Dashboards	Kubernetes
Kubernetes / Kubelet	Dashboard	Dashboards	Kubernetes
Kubernetes / Scheduler	Dashboard	Dashboards	Kubernetes
Kubernetes / Networking / Pod	Dashboard	Dashboards	Kubernetes
Kubernetes / Compute Resources / Pod	Dashboard	Dashboards	Kubernetes
Kubernetes / Networking / Cluster	Dashboard	Dashboards	Kubernetes
Kubernetes / Controller Manager	Dashboard	Dashboards	Kubernetes
Kubernetes / Persistent Volumes	Dashboard	Dashboards	Kubernetes
Kubernetes / Networking / Workload	Dashboard	Dashboards	Kubernetes
Kubernetes / Compute Resources / Node	Dashboard	Dashboards	Kubernetes
Kubernetes / Compute Resources / Cluster	Dashboard	Dashboards	Kubernetes
Kubernetes / Compute Resources / Workload	Dashboard	Dashboards	Kubernetes
Kubernetes / Networking / Namespace	Dashboard	Dashboards	Kubernetes
Kubernetes / Compute Resources / Namespace	Dashboard	Dashboards	Kubernetes
Kubernetes / Networking / Namespace	Dashboard	Dashboards	Kubernetes
Kubernetes / Compute Resources / Namespace	Dashboard	Dashboards	Kubernetes

Prometheus /metrics output from FastAPI (http_requests_total, process_memory, python_info)

10. Architecture Overview

The diagram below shows the full end-to-end MLOps pipeline from data ingestion to live monitoring.



11. Setup & Reproduction Instructions

11.1 Prerequisites

Python 3.11+

Docker Desktop

Azure CLI (az) with active subscription

kubectl configured to AKS

Helm 3.x

Git + GitHub account

11.2 Local Development

Clone:

```
git clone https://github.com/ParagSG/heart-disease-mlopsv1.git
```

Install:

```
pip install -r requirements.txt
```

Train:

```
python src/train.py
```

Serve:

```
uvicorn src.app:app --host 0.0.0.0 --port 8000
```

11.3 Docker

```
docker build -t heart-disease-api:latest .  
docker run -p 8000:8000 heart-disease-api:latest  
docker push paragmlopsacr.azurecr.io/heart-disease-api:latest
```

11.4 AKS Deployment

```
az aks get-credentials --resource-group mlops-rg --name mlops-aks  
kubectl apply -f k8s/deployment.yaml  
kubectl apply -f k8s/service.yaml  
kubectl get svc # wait for EXTERNAL-IP
```

11.5 Monitoring

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts  
helm install monitoring prometheus-community/kube-prometheus-stack \\\n  --namespace monitoring --create-namespace \\\n  --set grafana.adminPassword=MLOps@2026  
kubectl patch svc monitoring-grafana -n monitoring --type=json \\\n  -p='[{"op": "replace", "path": "/spec/type", "value": "LoadBalancer"}]'
```

12. Conclusion

This assignment demonstrates a complete, production-ready MLOps pipeline applied to a medical classification problem. From raw CSV data through structured EDA, reproducible preprocessing via sklearn pipelines, rigorous model training tracked in MLflow, containerised serving with FastAPI and Docker, automated CI/CD with GitHub Actions, and live deployment on Azure AKS with Prometheus/Grafana observability — every stage follows industry best practices.

The Logistic Regression model achieved a ROC-AUC of 0.9654 on the test set. The entire pipeline from a code push to a deployed, monitored API is fully automated and reproducible.

Key MLOps principles demonstrated: experiment reproducibility (MLflow), no training-serving skew (sklearn Pipeline), infrastructure as code (Kubernetes YAML + Helm), automated quality gates (GitHub Actions), and production observability (Prometheus + Grafana).

GitHub Repository

Repository URL: <https://github.com/ParagSG/heart-disease-mlopsv1>